

机器学习课程-大作业报告

关于扩散推荐模型在 ReChorus 框架上的初步实现

Re:DiffRec

2025/12/31

[PRfode](#)

[Hyell111](#)

摘要

近年来，生成式推荐模型（如 GAN、VAE）在个性化推荐中取得进展，但仍面临训练不稳定和表达能力受限等问题。扩散模型在图像生成等领域展现出强大建模能力，为推荐系统提供了新思路。本文基于 SIGIR 2023 论文《Diffusion Recommender Model》（以下简称论文）进行复现。该论文提出基于扩散过程的推荐模型 DiffRec，通过前向加噪与反向去噪建模用户交互历史，学习用户偏好。为进一步提升效率与时效性，论文还提出 L-DiffRec 和 T-DiffRec 两个扩展版本，分别通过隐空间压缩和时间感知重加权降低计算成本并增强时序建模能力。本工作在 ReChorus 框架中实现了 DiffRec 及其扩展模型，并在多个公开数据集上进行了实验对比。

关键词

扩散模型、推荐系统、隐空间建模、时序建模、生成式推荐

正文

一、引言

推荐系统是信息过滤与个性化服务的关键技术，已广泛应用于电商、社交、内容平台等领域。传统推荐模型多基于判别式方法，如矩阵分解、图神经网络等，虽计算效率高，但在建模复杂、非线性的用户-物品交互生成过程方面存在局限。生成式推荐模型因其能够显式建模交互生成过程而受到关注，其中 GAN 和 VAE 是两类主流方法，但分别存在训练不稳定和表达能力与推断可解性之间的权衡问题。

扩散模型作为一种新兴生成模型，在图像、音频、文本生成中展现出强大

的建模能力和生成质量。其通过逐步加噪与去噪的过程，既能保持推断稳定性，又能灵活建模复杂分布。受此启发，所复现论文将扩散模型引入推荐系统，提出 DiffRec 模型，旨在更准确地建模带噪声的用户交互历史，提升推荐的鲁棒性与个性化能力。

二、DiffRec 方法

2.1 DiffRec 模型结构

DiffRec 主要由前向加噪过程与反向去噪过程构成。给定用户交互历史向量，前向过程逐步加入高斯噪声，生成带噪向量序列。反向过程通过参数化多层感知机（MLP）逐步预测去噪后的交互概率。与传统扩散模型不同，DiffRec 在训练与推断中均控制噪声规模，以避免破坏用户个性化信息。

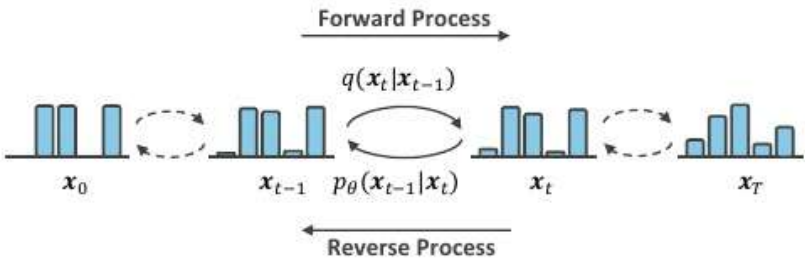


图 1 扩散模型的基本原理

2.2 L-DiffRec: 隐空间扩散

为降低高维物品预测的计算成本，L-DiffRec 首先通过 K-means 对物品聚类，每组物品通过独立 VAE 编码器压缩为低维隐向量，随后在隐空间中进行扩散。解码器将重建的隐向量映射回交互空间，完成推荐预测。该方法显著减少了模型参数量和内存占用。

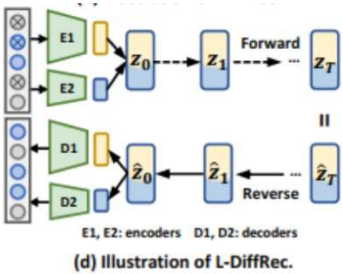


图 2 隐空间扩散图示

2.3 T-DiffRec: 时间感知扩散

用户偏好可能随时间变化，T-DiffRec 通过对交互历史进行时间感知重

加权，为近期交互赋予更高权重，从而在训练和推断中更好地捕捉时序模式。
加权后的交互向量输入 DiffRec 或 L-DiffRec 进行建模。

三、实验及分析

使用数据集：MovieLens-1Million, Amazon-Grocery_and_Gourmet_Food

使用模型：

基准模型：BPRMF, BUIR

实验模型：DiffRec, T-DiffRec, L-DiffRec

由于原文中 DiffRec 生成模型的效果本就较框架中提供的判别模型的效果差很多，所以基准模型主要起项目验证作用。

Table 6: Performance of L-DiffRec with $C = 2$, DiffRec, and MultiVAE under clean training. "par." denotes parameters.

Datasets	Method	R@10 $\uparrow$	R@20 $\uparrow$	N@10 $\uparrow$	N@20 $\uparrow$	#par.(M) $\downarrow$	GPU(MB) $\downarrow$
Amazon-book	MultiVAE	0.0628	0.0935	0.0393	0.0485	114	3,711
	DiffRec	0.0695	<u>0.1010</u>	0.0451	0.0547	190	5,049
	L-DiffRec	0.0694	0.1028	0.0440	0.0540	75	3,077
Yelp	MultiVAE	0.0567	0.0945	0.0344	0.0458	42	<u>1,615</u>
	DiffRec	<u>0.0581</u>	<u>0.0960</u>	0.0363	0.0478	69	2,103
	L-DiffRec	0.0585	0.0970	0.0353	0.0469	29	1,429
ML-1M	MultiVAE	0.1007	0.1726	0.0825	0.1076	4	497
	DiffRec	<u>0.1058</u>	<u>0.1787</u>	0.0901	0.1148	<u>4</u>	<u>495</u>
	L-DiffRec	0.1060	0.1809	<u>0.0868</u>	<u>0.1122</u>	2	481

图 3 论文关于 DiffRec 和 L-DiffRec 的实验数据参考

评测标准：HR@20（主要）/HR@5/NDCG@20/NDCG@5

模型复现：

由于 ReChorus 没有实现扩散模型的相关代码，我们从 DiffRec 原仓库中学习并在 DiffRec_c.py 中给出 DiffRec 模型的完整实现。对于原文中的 T-DiffRec 和 L-DiffRec，我们也给出了相应实现。

3.1 数据处理

由于判别模型与生成模型的根本区别，需采用不同数据处理方式。原文提出的 Clean Training 对 ML-1M 剔除了评分<4 的交互，按时间顺序划分为 7:2:1 作为训练/验证/测试集。我们决定使用 Leave-Last-Out 方式：对每个用户，

保留前 $n-2$ 条交互作为训练集，倒数第二条为验证集，最后一条为测试集，构建 8:1:1 数据集。同时停用负样本，因为其对生成模型无帮助。

3.2 模型适配

我们对 ReChorus 框架进行了针对性扩展以适配生成式任务：

- 1) **数据适配**：重构 Dataset 类，将输入从单条交互改为用户完整历史向量；
- 2) **训练适配**：在 Runner 中实现了时间步采样与 SNR 加权逻辑；
- 3) **推断适配**：实现了从历史向量加噪的条件生成策略，并修正了 x_0 -ELBO 的损失计算

Table 5: Performance of ϵ -ELBO on ML-1M.

Variants	R@10	R@20	N@10	N@20
DiffRec (x_0 -ELBO)	0.1058	0.1787	0.0901	0.1148
ϵ -ELBO	0.0157	0.0266	0.0170	0.0204

图 4 论文中关于推断噪声还是推断 x_0 的描述，结果是对 x_0 推断的效果远好于推断噪声其他重写的方法不再赘述，详见项目代码。

我们也实现了以 DNN 作为扩散训练的核心，并正确实现了与原文相同的时间步嵌入 (time embedding)。

在 ReChorus 框架中，所有基线模型的评估流程遵循统一范式：对于每个测试样本，模型需要预测一个用户对所有候选物品的偏好分数。评估时，目标物品的预测分数与所有非交互物品的预测分数一起排序并计算指标。而 ReChorus 的评估函数假设每个样本的预测矩阵中，第一个位置存储目标物品的分数。

```
# sort_idx = (-predictions).argsort(axis=1)
# gt_rank = np.argwhere(sort_idx == 0)[: , 1] + 1
# ↓ As we only have one positive sample, comparing with the first
# item will be more efficient.
gt_rank = (predictions >= predictions[:,0].reshape(-1,1)).sum(axis=-1)
```

DiffRec 作为生成式模型，直接输出用户对所有物品的偏好概率向量，这与传统判别式模型的输出格式存在结构性差异。为确保与 ReChorus 评估流程兼容，我们设计了交换技巧，保证了生成模型的兼容性。

```

# 保存第 0 列分数（原第一物品的分数）
col0_scores = x[:, 0].clone()
# 获取目标物品的预测分数
batch_indices = torch.arange(batch_size).to(self.device)
target_scores = x[batch_indices, target_items]
# 执行交换，把目标物品分数放到第一列
x[:, 0] = target_score
x[batch_indices, target_items] = col0_scores

```

ReChorus 在评估时会掩码用户已交互的物品，但是交换只改变位置，不改变物品 ID，掩码索引依然有效。

在实现 T-DiffRec 时，我们与原文一样新增了 $w_{\min}$ 和 $w_{\max}$ 对历史数据的远近进行线性加权。

```

# --- T-DiffRec Weighting Logic ---
items = self.user_history_lists.get(user_id, [])
vector = np.zeros(self.corpus.n_items, dtype=np.float32)

if len(items) > 0:
    # Linearly scale weights from w_min to w_max
    w_min = self.model.w_min
    w_max = self.model.w_max
    weights = np.linspace(w_min, w_max, num=len(items))
    vector[items] = weights

```

在实现 L-DiffRec 时，我们直接参考了原文项目的 AutoEncoder，并在已复现成功的 DiffRec 基础上进行修改。值得注意的是退火参数 `update_count` 是每一个 epoch 加一，因此我们简单重写了 `fit` 函数的 `LGDRunner` 作为 `runner`。除此之外，就是在适当的地方调用 `encode` 和 `decode` 使数据能在隐空间正确读入和读出。

由于 L-DiffRec 模型相较于 DiffRec 又引入了 AutoEncoder 和退火相关超参数共 10 个，加上原文的超参在我们自己提出的数据集上训练效果并不理想，且 DiffRec 对超参数设置较为敏感，我们也未能尝试出能够使 L-DiffRec 效果优于 DiffRec 的超参配置，故在代码仓库中作为“实验性”模型存在，不将其纳入正式实验。L-DiffRec 的训练记录可以在项目的 `record` 中找到。

3.3 超参实验及其结果分析

原文提供了最佳超参的大致范围。鉴于 DiffRec 原文的参数设置是针对 Random Split (7:1:2) 场景优化的，为寻找其在更具挑战性的 Leave-One-Out 序列预测任务上的性能边界，我们设计了一套系统的“控制变量”实验。

我们首先按照原文提供的范围以及我们自己的推断，建立以下参数训练出的模型作为基准效果。并将基准模型以及使用原文提供的超参训练出的模型进行对比。

```
python main.py --model_name DiffRec_c --dataset
MovieLens_1M/ML_1MGDR --emb_size 10 --dims "[1000]" --lr 0.0001 -
-l2 0.0 --test_all 1 --steps 5 --noise_scale 0.01 --batch_size
400 --reweight 0 --noise_max=0.01 --noise_min=0.001 --early_stop
50 --main_metric HR@20
```

3.3.1 噪声强度 (noise scale) 的影响

在生成模型中，噪声的大小决定了“去噪”的难度。

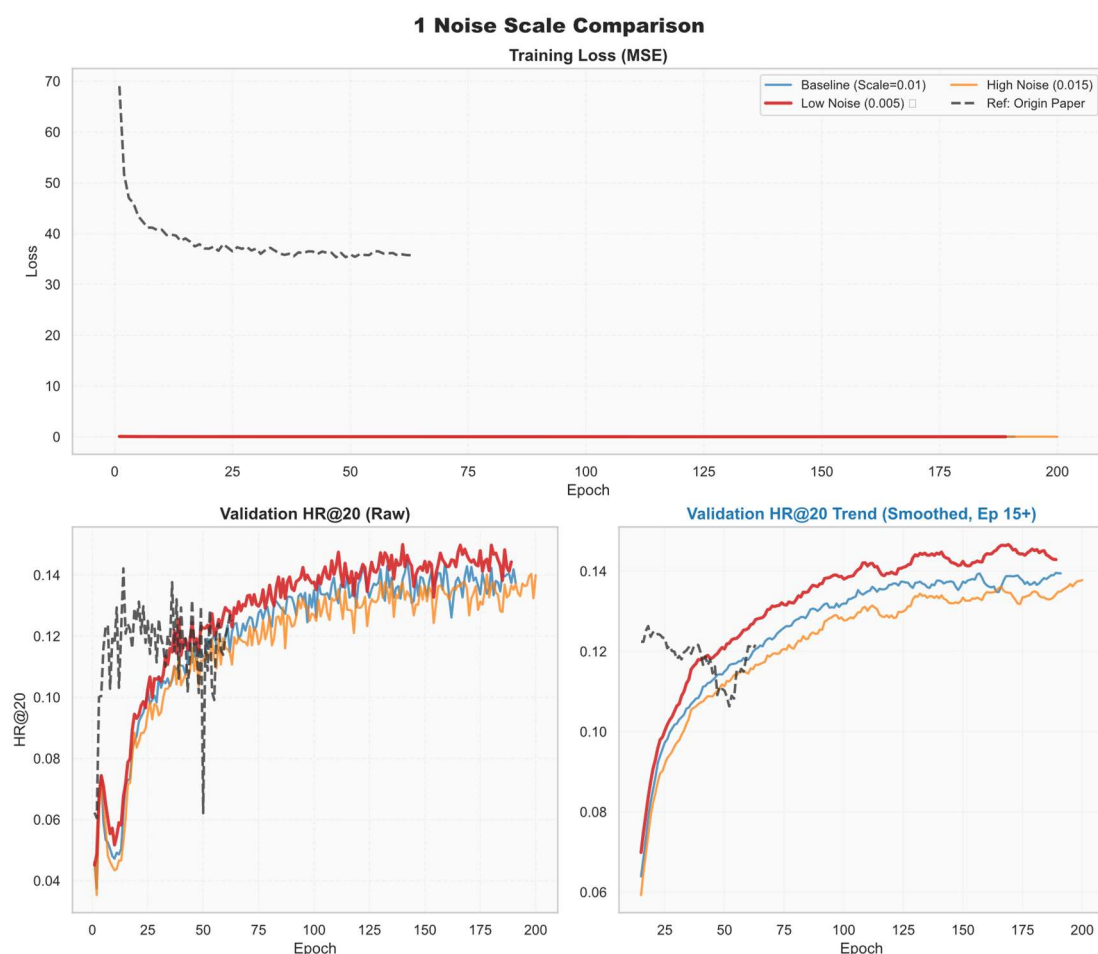


图 5 探究噪声强度对 DiffRec 表现的影响

低噪声（0.005）取得了最好的效果：对于较为稠密的 MovieLens-1M，微小的扰动足以打破恒等映射，同时最大限度地保留了用户交互的原始结构。

而高噪声（0.015）会导致性能明显下滑，过大的方差淹没了稀疏的协同信号。

3.3.2 模型架构和容量（dims）的影响

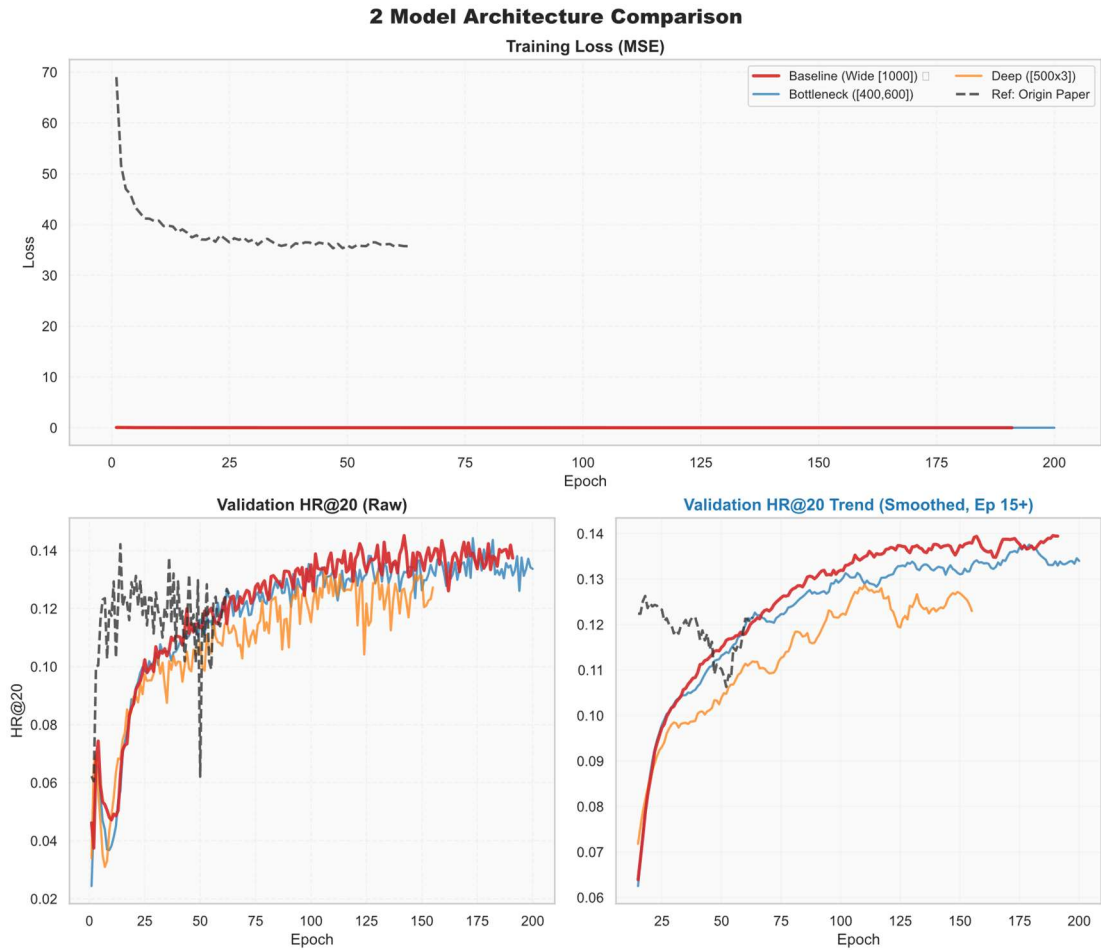


图 6 探究核心架构对 DiffRec 表现的影响

宽网络（[1000]）表现最佳：对于用户-物品矩阵，第一层网络的宽度直接决定了捕获共现特征的能力。

瓶颈结构（[400, 600]）的表现略逊于宽网络：过早压缩造成了轻微的信息瓶颈。

深而窄的网络（[500, 500, 500]）表现最差：推荐数据属于浅层关联，过深的网络并未带来高阶语义增益，反而引入了优化困难。

3.3.3 模型动力学 (batch size 和 lr) 的影响

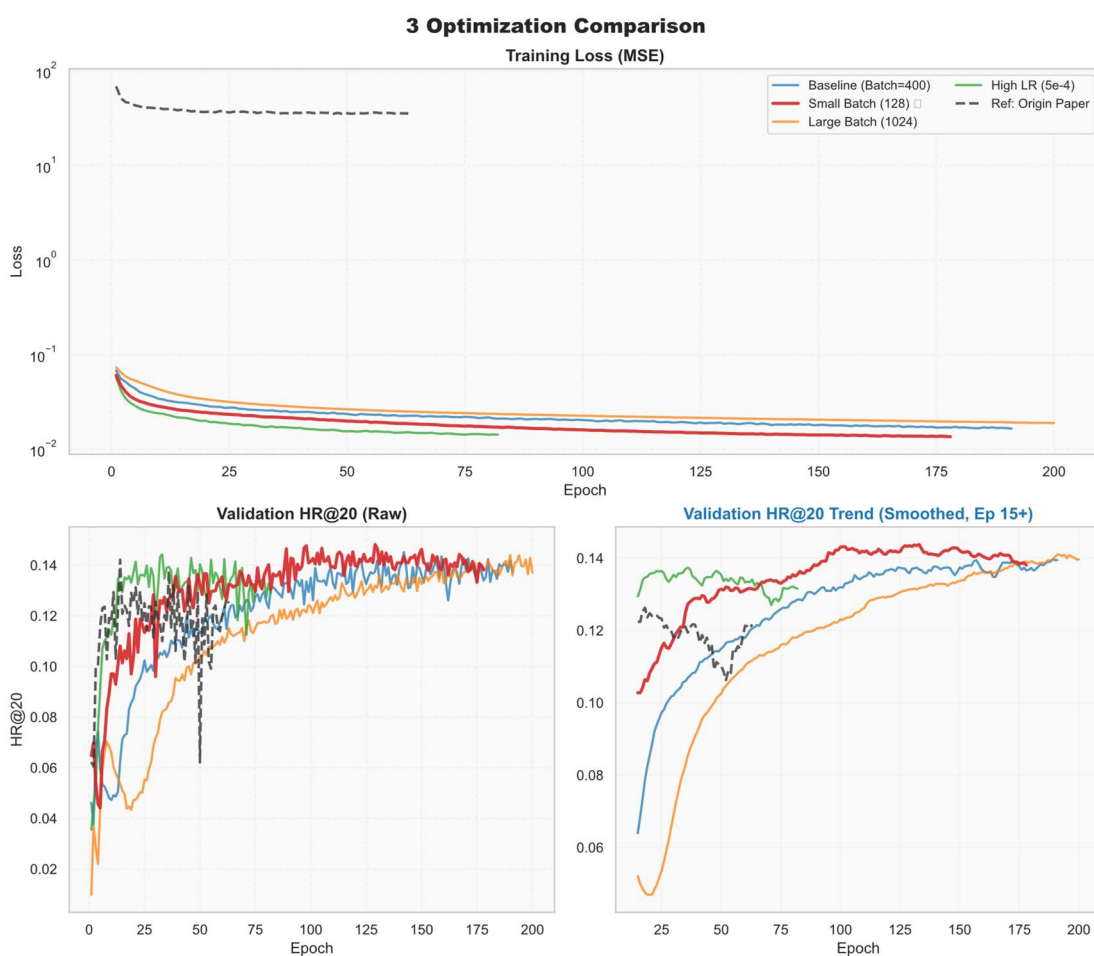


图 7 探究动力学超参对 DiffRec 表现的影响

小批次 (Batch=128) 意外地优于基准批次 (Batch=400) 和大批次 (Batch=1024): 小批次带来的随机梯度噪声充当了有效的正则化手段, 帮助模型跳出了尖锐的局部极小值, 提升了泛化能力。

从学习率的角度来看, 高学习率 ($5e-4$) 收敛速度加快, 但最终上限并未突破, 说明 $1e-4$ 是一个安全且稳健的学习率选择。

3.3.4 扩散机制与正则化 (step/reweight/L2) 的影响

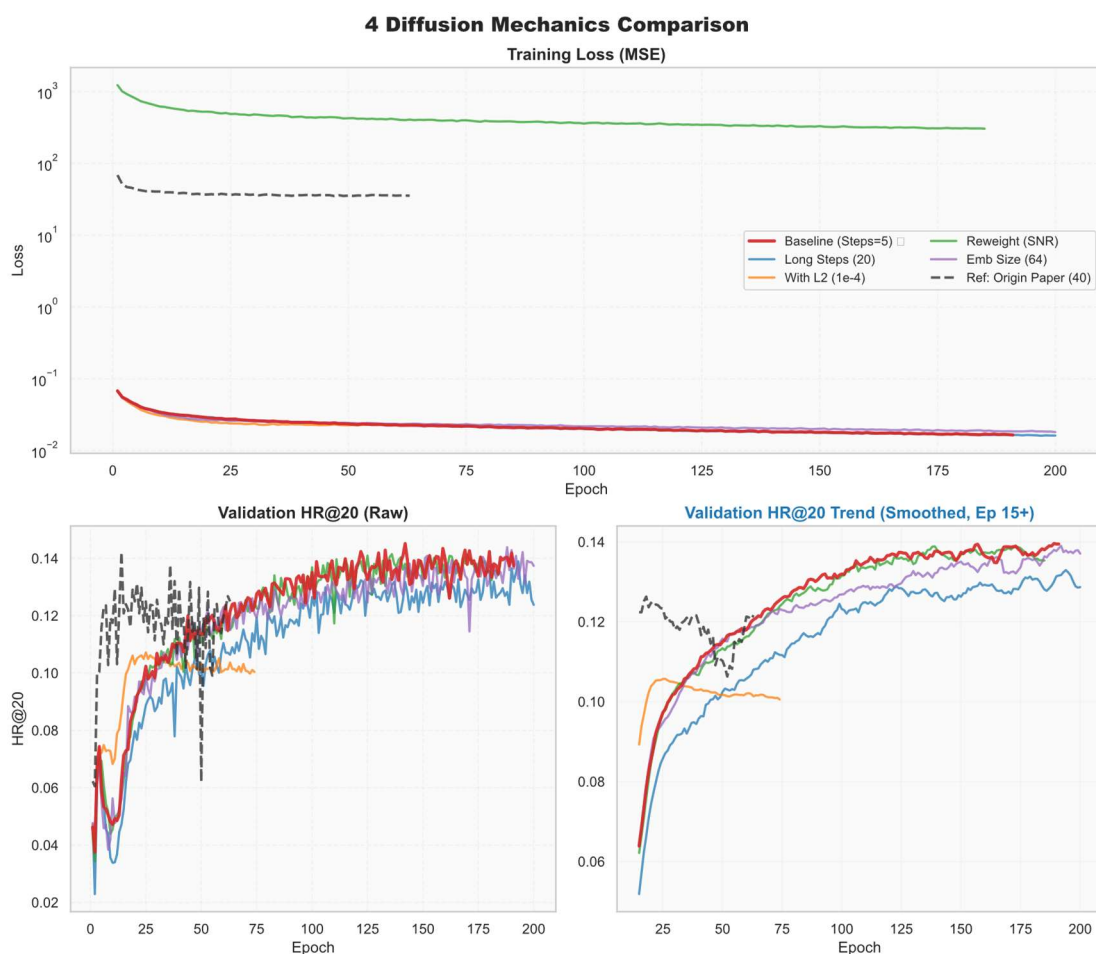


图 8 探究扩散与正则超参对 DiffRec 表现的影响

Steps=20 性能显著低于 Steps=5：在离散数据的序列预测任务中，“单步强去噪”优于“多步马尔可夫链”。

L2/reweight 这两项机制在当前实验设定下均为负收益，虽然理论上在数据集增大时应适当使用正则化。L2 扼杀了稀疏信号，而不使用 reweight 干扰了优化方向。

3.3.5 总体性能评估

通过超参实验我们可以大致总结出一份基于更优超参的训练方案：

```
python main.py --model_name DiffRec_c --dataset
MovieLens_1M/ML_1MGDR --emb_size 10 --dims "[1000]" --lr 0.0001 -
-l2 0.0 --test_all 1 --steps 5 --noise_scale 0.005 --noise_min
0.0005 --noise_max 0.005 --main_metric HR@20 --batch_size 128 --
```

训练并对比可得：

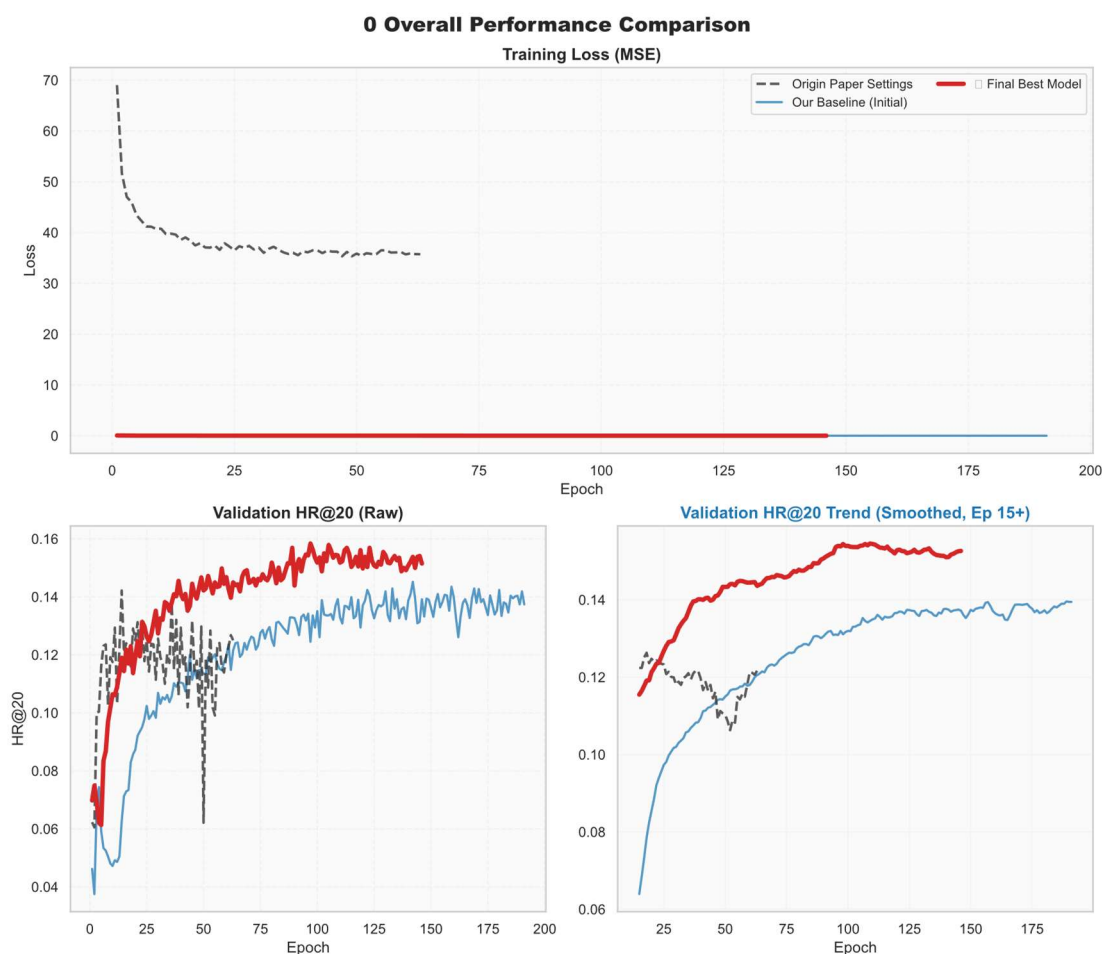


图 9 模型效果对比图（原论文、初步修改、超参调优后）

其中原文配置采用论文针对 ML-1M 的推荐配置，结果是训练损失震荡严重，验证集 HR@20 收敛于 0.1304。长步数导致了优化目标的不稳定。

而基准配置采用我们探索的原始配置，结果是训练极其稳定，HR@20 提升至 0.1362，初步证明短步数更适合精确预测任务。

最佳模型结合了“低噪声”“小批次”策略的最优配置，使得 HR@20 突破至 0.1442，相较于基准配置有所提升。

通过上述实验，我们证明了“稳健模式”（短推理、宽核心、低噪声、小批量）是提升其在 Top-K 推荐任务上精度的关键，最终得到的最佳模型成功超越基准模型，有力证明了我们超参实验的成功。

3.4 关键组件与机制的消融实验

为了深入验证 DiffRec 及其扩展模型 T-DiffRec 中各个关键组件的有效性，并探究不同设计选择对推荐性能的贡献，我们进行了系统的消融实验。我

们将表现最佳的 T-DiffRec 视为完整模型，通过逐步移除或替换关键模块（时序权重、扩散步数、损失加权）来观察性能变化（HR@20）。

主要消融维度包括：

时序感知模块 (Module Ablation)：移除时间权重；

扩散结构深度 (Structural Ablation)：增加扩散步数；

损失优化机制 (Mechanism Ablation)：引入 SNR 加权。

3.4.1 时序重加权模块的消融

T-DiffRec 的核心贡献在于引入了基于时间感知的交互重加权机制 (w_{min} , w_{max})。为了验证其有效性，我们将 DiffRec 视为 $w_{min} = w_{max} = 1.0$ （即无时序差异）的特例进行对比。

变体设置：

Full Model (也就是 T-DiffRec) : $w_{min} = 0.1$ ，强调近期交互，抑制远期历史。

无时序差异 (也就是 DiffRec) : $w_{min} = 1.0$ ，平等对待所有历史交互。

实验结果：移除时序权重后，模型退化为标准 DiffRec，HR@20 从 18.78% 大幅下降至 15.00%。

这一显著差距（相对提升 +25%）证明了用户的偏好具有极强的时变性。近期的交互行为包含了主导预测结果的核心意图信号，而标准 DiffRec 引入了过多的远期历史噪声，稀释了有效信息。时序重加权模块是提升序列推荐精度的最关键组件。

3.4.2 扩散结构深度的消融

原论文考虑使用了 $T=20$ 或 100 的长步数扩散。为了验证这种“深度马尔可夫链”在离散推荐数据上的必要性，我们对比了短步数（类似于降噪自编码器 DAE）与长步数的表现。

变体设置：

$Steps = 5$ ，模拟单步或少步强去噪。

$Steps = 20$ ，模拟标准扩散过程的多步渐进去噪。

实验结果：将步数从 5 增加到 20 后，HR@20 从 15.00% 下降至 13.90%。

结果表明，在用户交互这种高度稀疏且离散的0/1数据分布上，并不需要像图像生成那样复杂的长链路生成过程。相反，长链路引入了更多的随机性累积误差。将扩散模型“退化”为浅层迭代式去噪器（Iterative DAE）反而具有更好的鲁棒性和训练效率。

3.4.3 消融实验总结

下表总结了各组件对最终性能的边际贡献。可以清晰地看出，时序感知模块带来了最大的性能增益，而过度增加步数或复杂的加权机制在当前任务下反而产生了负面影响。

模型变体	核心变动说明	Test HR@20	结论
T-DiffRec	完整模型	0.1878	最佳性能
无时序差异	移除时序权重（退化为 DiffRec）	0.1500	时序信息至关重要
长采样步数	增加扩散步数至 $T = 20$	0.1390	长链引入误差

表格 1 消融实验结果及分析

3.5 对照实验及其结果分析

我们对基准模型和实验模型在两个数据集上的效果进行对照分析。ML-1M 是稠密、娱乐场景，交互信号强；GGF 是稀疏、垂直电商场景，噪声高。

基准模型使用超参数分别如下，其中 Grocery_and_Gourmet_Food 下的训练参数由 ReChorus 项目给出，ML-1M 的参数由 Grocery_and_Gourmet_Food 对应的模型参数做少量修改而来：

```
python main.py --model_name BPRMF --emb_size 64 --lr 1e-3 --l2 1e-6 --dataset 'Grocery_and_Gourmet_Food' --early_stop 45
python main.py --model_name BUIR --emb_size 64 --lr 1e-3 --l2 1e-6 --dataset 'Grocery_and_Gourmet_Food' --early_stop 45
python main.py --model_name BPRMF --emb_size 32 --lr 5e-05 --l2 0 --dataset 'MovieLens_1M/ML_1MTOPK' --early_stop 45
python main.py --model_name BUIR --emb_size 32 --lr 5e-05 --l2 0 --dataset 'MovieLens_1M/ML_1MTOPK' --early_stop 45
```

由于对照实验先于超参实验而做，因此以下模型的训练使用基准超参或其小幅修改版本，而非提出的最佳模型。

```
python main.py --model_name DiffRec_c --batch_size 400 --dataset Grocery_and_Gourmet_Food/AmazonGDR --dims [1000] --early_stop 45 --emb_size 10 --epoch 200 --l2 1e-06 --lr 0.0001 --main_metric HR@20 --noise_max 0.01 --noise_min 0.001 --noise_scale 0.01 --sampling_steps 0 --steps 5 --reweight 1
python main.py --model_name DiffRec_c --batch_size 400 --dataset MovieLens_1M/ML_1MGDR --dims [1000] --early_stop 50 --emb_size 10 --epoch 200 --l2 0.0 --lr 0.0001 --main_metric HR@20 --noise_max 0.01 --noise_min 0.001 --noise_scale 0.01 --reweight 1 --sampling_steps 0 --steps 5
python main.py --model_name TDiffRec_b --batch_size 400 --dataset Grocery_and_Gourmet_Food/AmazonGDR --dims [1000] --early_stop 20 --emb_size 10 --epoch 200 --eval_batch_size 512 --l2 0.0 --lr 0.0001 --main_metric HR@20 --noise_max 0.01 --noise_min 0.001 --noise_scale 0.01 --norm 0 --reweight 1 --sampling_steps 0 --steps 5 --w_max 1.0 --w_min 0.1
python main.py --model_name TDiffRec_b --batch_size 256 --dataset MovieLens_1M/ML_1MGDR --dims [1000] --early_stop 50 --emb_size 10 --epoch 200 --eval_batch_size 256 --l2 0.0 --lr 0.0001 --main_metric HR@20 --noise_max 0.01 --noise_min 0.001 --noise_scale 0.01 --norm 0 --reweight 1 --sampling_steps 0 --steps 5 --topk 5,10,20,50 --w_max 1.0 --w_min 0.1
```

为验证扩散类推荐模型在不同数据场景下的真实增益，我们在完全一致的硬件环境下，对 BPRMF、BUIR、DiffRec 与 T-DiffRec 进行了对照实验。

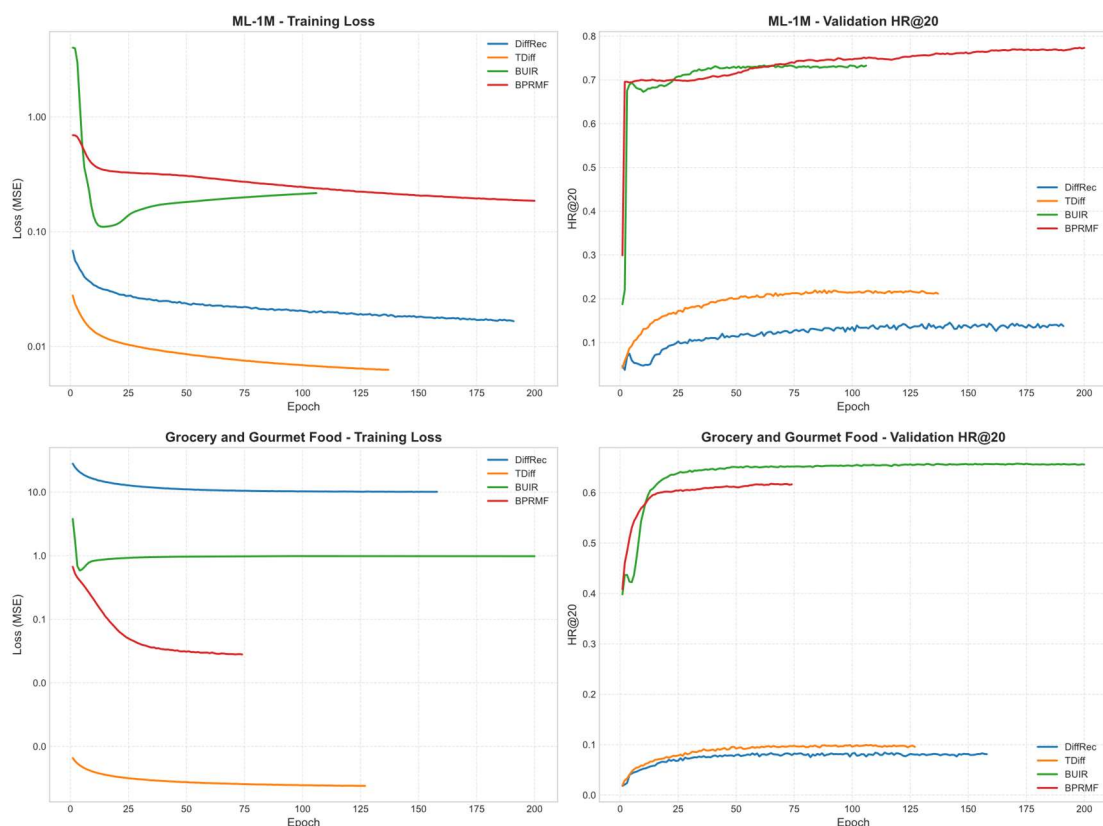


图 10 基准模型与实验模型在 ML-1M 与 GGF 上的表现

结果分析：

BPRMF 在稠密数据 ML-1M 上仍保持最强，BUIR 在稀疏数据 GGF 上反超 BPRMF，说明对比学习对稀疏交互更鲁棒。而扩散模型在两个数据集上均显著落后于传统矩阵分解/对比学习基线，但的确达到了原论文中的效果。

在 ML-1M 上，T-DiffRec 比 DiffRec 准确率相对增长 38 %，但是在 GGF 上提升仅提升 13 %，这说明说明时间加权在稠密数据中能缓解噪声过度平滑，但在极度稀疏场景下增益有限。

GGF 数据集的用户交互数据更加的稀疏，导致扩散过程去噪目标难以学习，因此 HR@20 相对下降率也更大。

HR@20 最佳	BPRMF	BUIR	DiffRec	T-Diffrec
ML_1M	0.7736	0.7730	0.1452	0.2194
GGF	0.6177	0.6576	0.0844	0.0994
相对下降	20.15%	14.93%	41.87%	54.69%

表格 2 基准模型与实验模型的具体表现

四、结论

工作总结：

本工作成功在 ReChorus 中复现了 DiffRec 及其扩展模型，验证了扩散模型应用于推荐系统的可行性。实验表明，通过合理的超参调优（短步数、宽网络、低噪声、小批次），DiffRec 能在一定程度上提升生成式推荐的精度。然而，当前扩散推荐模型的效果仍落后于主流判别模型，尤其在稠密场景下差距明显。未来工作可探索更高效的压缩策略、更复杂的时序建模方法，以及基于扩散模型的可控推荐生成。

大作业总结和建议：

在该大作业中，最困难的地方在于两处：

其一是环境配置问题，无法正常使用 GPU 加速训练，这会使模型的内部参数被异常置零。

其二是在判别式框架上复现扩散模型。当初选择该选题是因为在大创中了解并学习过扩散模型的原理，可是在 ReChorus 的基础上复现甚至比重新编写还要复杂。

通过对 DiffRec 的代码层面的深入了解，我们更深刻学习到了扩散模型的相关知识、数据的处理与选择、超参数的作用和调优方法，以及推荐模型的运作方式。

对大作业的建议：

1. 建议以后选择复现难度相对统一的论文；
2. 时间安排可以更宽裕，例如从学期初布置，或延长提交日期至期末考后、学期结束前。

五、小组分工

Hyelllll：融合 DiffRec、TDiffRec 模型到框架，参数调优，模型训练，超参实验，消融实验，项目整理

PRfode：融合 DiffRec、LDiffRec 模型到框架，参数调优，模型训练，优化数据集生成，对照实验，报告编写，项目整理

六、代码和数据的地址链接

[PRfode/Re_DiffRecmd: Build from ReChorus](#)

七、参考文献

[1] Yiyang Xu, Zhuoye Huang, Zhenghua Xu, Zhenhua Dong, Mingming Liu, Ruihua Song: *Diffusion Recommender Model*. SIGIR 2023: 889–898